

Hausaufgabe 4: RISC-V Konventionen

Aufgabe 1: RISC-V Konventionen

1. Mögliche Konventionen (Allgemein)

Registersicherung:

1. Caller-Save: Das aufrufende Unterprogramm sichert temporäre Register vor der Benutzung auf dem Stack und stellt sie danach wieder her.
2. Callee-Save: Das aufgerufene Unterprogramm sichert Register, die es verändern möchte (z.B. s0 - s11), und stellt sie vor dem Rücksprung wieder her. (Siehe Vorlesung 10.72)

Alignment: Daten und Instruktionen müssen an Adressen liegen, die:

1. Ein Vielfaches einer Zweierpotenz sind.
2. Ein Vielfaches der Größe des Datums sind (Natural Alignment).[(Siehe Vorlesung 08.93)]

Stack:

1. Wachstumsrichtung: Entweder von oben nach unten (High to Low) oder von unten nach oben.
2. Parameter: Können auf dem Stack vor oder nach der Rücksprungsadresse liegen.

Parameterübergabe:

1. Call-by-Value: Ein Wert (Kopie) wird übergeben.
2. Call-by-Reference: Eine Speicheradresse (Referenz) wird übergeben.

(Anmerkung: „Call-by-Name“ ist ein historisches Konzept (z. B. aus ALGOL 60). Das ist veraltet, weil es sogenannte ‚Thunks‘ (Hilfsfunktionen) benötigt, um Parameter bei jedem Zugriff neu zu berechnen. Es ist ineffizient und die RISC-V ABI unterstützt es nicht.)

2. Konkrete RISC-V Konventionen

Registersicherung & Register-Tabelle: Die genaue Aufteilung der Register ist im **RISC-V Instruction Set Manual** (Waterman et al., 2019) definiert. [Hier ist die vollständige Übersicht der Register und ihrer Verantwortlichkeiten:](#)

Register	ABI Name	Beschreibung	Saver
x0	zero	Hard-wired Zero	–
x1	ra	Return Address	Caller
x2	sp	Stack Pointer	Callee
x3	gp	Global Pointer	–
x4	tp	Thread Pointer	–
x5-x7	t0-t2	Temporaries	Caller
x8	s0/fp	Saved Register / Frame Pointer	Callee
x9	s1	Saved Register	Callee
x10-x11	a0-a1	Function Arguments / Return Values	Caller
x12-x17	a2-a7	Function Arguments	Caller
x18-x27	s2-s11	Saved Registers	Callee
x28-x31	t3-t6	Temporaries	Caller

Tabelle 1: RISC-V Calling Convention (nach Waterman et al., 2019)

Wichtige Ergänzungen zur Tabelle:

- Saved Registers (Callee): Dazu gehören s0 - s11 sowie zwingend der Stack Pointer (sp). Das Unterprogramm muss den Stack am Ende im gleichen Zustand hinterlassen. Auch die Return Address (ra) muss gesichert werden, falls das Unterprogramm selbst weitere Funktionen aufruft (Nicht-Blatt-Funktion).
- Temporary Registers (Caller): t0 - t6 und a0 - a7 dürfen vom Unterprogramm überschrieben werden.

Alignment:

1. Instruction Alignment: Vielfache von 4 (bzw. 2 bei Compressed Instructions).
2. Data Alignment: Natural Alignment wird empfohlen.
3. Stack Alignment: **Der Stack Pointer (sp) muss beim Eintritt in eine Funktion zwingend 16-Byte aligned sein.** Dies dient der Kompatibilität mit 128-Bit Datentypen (long double) und Vektor-Erweiterungen (RV128 / Vector Extension).

Stack:

1. Richtung: Wächst von oben nach unten (High Address to Low Address).
2. Layout: Der Stack Frame enthält typischerweise (von oben nach unten): Rücksprungadresse (ra), gesicherte Register (sX), und lokale Variablen.

Parameterübergabe: Gemäß RISC-V Calling Convention:

1. Register: Die ersten 8 Argumente werden in a0 bis a7 übergeben.
2. Stack (Spilling): **Argumente, die nicht mehr in die Register passen (ab dem 9. Argument), werden auf dem Stack übergeben.**
3. Große Daten: Argumente, die größer als ein Wort sind (z.B. große Structs), werden oft „by-reference“ übergeben (Pointer im Register).
4. Rückgabewerte: Liegen in a0 (und a1 für 64-Bit Ergebnisse bei RV32).